

Randomized Cassandra Consistency Experiments

Registered redesign plan and reproducibility protocol

Randomized Cassandra Consistency Experiments

****Detailed redesign plan | 12 September 2026****

****Status:** planned, not executed. ****** This document specifies the replacement experiments. The existing 300-trial results and PDFs describe the previous fixed-coordinator design and must not be presented as results of this redesign.

1. Objective and meaning of random access

Investigate read-your-writes (RYW), monotonic reads (MR), monotonic writes (MW), and writes-follow-reads (WFR) when an application cannot choose the database node handling a request. Each measured read and write must receive an independent random routing decision.

The customer-facing API will expose ``read(key, consistency)`` and ``write(key, value, consistency)``, with no node parameter, preferred-node hint, sticky session, or knowledge of the current fault. Test administration can address individual containers to install the database, inspect health, or inject faults; those actions are separate from application operations.

****Important Cassandra distinction:**** a randomly selected node is the request's coordinator. Cassandra determines which replicas receive a write or serve a read. With three nodes and replication factor three, all three nodes are replicas for every test key. A write at ONE does not mean Cassandra deliberately writes to only one node; ONE controls the required acknowledgements. A read coordinated by n2 is not necessarily served from n2's local data. [1]

We will therefore describe the experiment as ****random coordinator routing****, not random single-replica storage. Record the actual coordinator, but do not claim to know the serving replicas unless separate tracing establishes them.

Uniform random routing is an explicit experimental policy. Customer lack of node control does not imply that a production driver is uniformly random: driver policies may consider token ownership, locality, and health. This study models a transparent random routing layer, not every production driver's behavior. [2]

The routing layer may maintain one backend connection per endpoint. Pinning such a backend connection implements a route already chosen at random; the application cannot select that backend. This separation must be visible in code and explained in the report.

2. Deployment retained

- Three Apache Cassandra 5.0.9 containers: n1, n2, n3.
- One datacenter, ``dc1``, with `NetworkTopologyStrategy` and replication factor 3.
- A separate Python client container on the same Docker bridge network.
- Existing pinned Docker base images and recorded runtime/package versions.
- Native CQL on TCP 9042; internode traffic on TCP 7000/7001.
- A dedicated fault chain inside each database container; no host firewall changes.
- Hinted handoff disabled during the controlled experiments; no scheduled anti-entropy repair. State this departure from default settings prominently. [3]

- Main table: ``read_repair=BLOCKING`, `speculative_retry=NONE``. The focused repair comparison additionally uses ``read_repair=NONE``. [4]

3. Randomization protocol

3.1 Independent routing per operation

1. At the start of each stable workload phase, the routing layer probes all three endpoints from the client container. Use a native CQL readiness check, not only a TCP port-open check. Save each probe's result and timestamp.
2. Build the candidate set using only client-observed endpoint reachability. Do not pass the fault victim, partition membership, replica versions, or experiment model into the router.
3. For every application operation, draw uniformly from that candidate set with replacement. A later request may legitimately choose the same node again; do not force coordinator changes.
4. Save the candidate set, seed identifier, draw index, selected endpoint, and actual coordinator with the operation record.
5. Do not reroute an operation because it returned stale data, Unavailable, or a timeout. Disable driver retries, consistency downgrades, and speculative executions for measured operations. [2]

The candidate set is refreshed between phases; faults remain fixed within a phase. This is a study of operation behavior after fault detection, not a measurement of continuously changing production service discovery. If a new failure occurs during a phase, retain the resulting error and health evidence.

During normal operation, all three endpoints should be eligible. After one node is killed and detection completes, the two surviving endpoints should be eligible. During an internode partition, all three CQL endpoints remain reachable and must stay eligible: the routing layer must not silently exclude the isolated side merely because it cannot reach a quorum.

3.2 Seeds and independent random streams

Generate a fresh seed from operating-system randomness for a new measured run, and save it before executing requests. Derive separate seeds for routing, case order, fault victim, and read-repair topology choices. Record the seed derivation and Python version.

A replay option may reuse the saved seed to reconstruct the intended choices. It cannot guarantee identical responses, timing, or health observations. Reproducible pseudorandom choices and nondeterministic distributed execution are compatible.

No seed search, outcome-dependent rerolling, or rejection of same-node sequences is permitted. A run may select some nodes more often than others; report the imbalance instead of correcting the measured data after the fact.

3.3 Routing evidence and audit

Report coordinator selection counts by node, scenario, and read/write operation type. Also report the write-to-read coordinator transition matrix, the fraction of consecutive operations that used different coordinators, and the number of trials spanning partition sides.

These are descriptive coverage measures. Ten draws need not look uniform. Do not use an arbitrary balance threshold to discard an otherwise valid run. Candidate membership and the uniform-selection implementation are checked separately from observed frequency.

3.4 Stable workload assumptions

Use fresh keys per trial, one logical writer per key, sequential operations, no TTLs or deletions, and explicit increasing mutation timestamps except in the timestamp control. Initialize each row to `(a=0,b=0)` at ALL while all nodes are healthy. Initialization also uses random routing, but is recorded separately from measured operations.

Logical clients can use different backend connections over time. Their ordered histories and client identifiers define the sessions; the driver connection is not the session definition.

4. Experiment matrix and counting

Configuration notation is ****write consistency / read consistency****. Test ONE/ONE, QUORUM/ONE, ONE/QUORUM, QUORUM/QUORUM, and ALL/ALL.

| Scenario | Configurations | Models per configuration | Trials per model/configuration | Main trials |

|---|---|---|---|---|

| Normal operation | All five | RYW, MR, MW, WFR | 10 | 200 |

| One node unavailable | All five | RYW, MR, MW, WFR | 10 | 200 |

| Internode network partition | All five | RYW, MR, MW, WFR | 10 | 200 |

| Main total | 5 | 4 | 10 in each scenario | 600 |

| Additional experiment type | Trials |

|---|---|

| Randomized read-quorum changes with BLOCKING repair | 10 |

| Randomized read-quorum changes with NONE repair | 10 |

| Decreasing-timestamp control with random coordinators | 10 |

| Additional total | 30 |

| Entire planned run | 630 |

A ****main trial**** is one complete scheduled logical history for one model, configuration, and scenario, using one fresh key. Each model therefore has 150 main trials: 3 scenarios x 5 configurations x 10 repetitions. There are 1,800 scheduled main application operations: 600 per scenario. Setup and health probes are not included in these counts.

4.1 Ten rounds and fault episodes

Run ten rounds. In each round, execute one block for each main scenario, in a randomized order. A block contains one trial for each of the 20 model/configuration combinations, also in randomized order. Initialize that block's keys while the cluster is healthy, then establish the assigned fault, if any.

Each round independently samples the node to kill and the node to isolate. Restore and verify the cluster between scenario blocks. This produces ****10 kill/restart episodes and 10 main partition/heal episodes****, rather than applying one fault once and reusing it for every repetition.

The 20 trials within a block share a fault episode. Their distinct keys do not make them statistically independent fault injections. Retain round and episode identifiers, summarize variation by episode, and do not report 600 independent fault realizations.

The read-repair comparison uses another 10 topology-changing episodes. Each episode contains one BLOCKING trial and one NONE trial, using separate keys and independent routing draws. Their shared topology must be disclosed. Thus the plan contains 10 crash episodes and 20 network-fault episodes in total; individual repair episodes contain several partition transitions.

4.2 Minimum trials and unsuccessful histories

Ten means ten measured attempts per case, not ten successful outcomes. ALL may be unavailable throughout a fault scenario, making ten successful trials impossible. Do not hide these attempts or rerun them until they pass.

A failed initialization or unverified fault is a setup failure, not a valid measured trial. Preserve the failed setup and mark the run incomplete until all required measured attempts exist. Any corrective execution must have a new attempt

identifier and remain distinguishable from the failed setup. Do not replace a valid but inconvenient measured outcome.

5. Model-specific workloads and verdicts

Every W or R below gets a fresh random coordinator decision, including producer and observer operations. Each trial initializes both cells at ALL first; W and R then use the selected case's consistency levels.

| Model | Scheduled application operations | Violation witness |

|---|---|---|

| RYW | Client W(a=1); same client R(a,b) | Acknowledged write followed by successful read with a=0 or the row missing |

| MR | Producer W(a=1); client R1(a,b); same client R2(a,b) | R1 observes a=1, then R2 observes a=0 or the row missing |

| MW | Client W(a=1); same client W(b=1); observer R(a,b) | Both writes acknowledged and observer sees (a=0,b=1) |

| WFR | Producer W(a=1); client R(a,b); same client W(b=1); observer R(a,b) | Dependency read sees a=1; writes acknowledged; observer sees (a=0,b=1) |

For MW/WFR, read both cells in one SELECT. Two separate observer reads could introduce their own read-order artifact. Explicit role and logical-client identifiers distinguish the WFR producer, dependent client, and observer.

MW/WFR are searches for out-of-order dependency visibility. They do not reconstruct every replica's internal application order. A missing predecessor with a visible successor is useful counterexample evidence; an observer seeing both values does not prove a general ordering guarantee.

5.1 Classification rules

- ****Violation:**** the defined witness appears in a valid history with the required acknowledged operations.
- ****No violation observed:**** the necessary observations complete and the witness is absent. This is a finite observation, not a proof.
- ****Inconclusive — operation error:**** a required request is unavailable, times out, or fails. Preserve the exception, CL, and any server response counts.
- ****Inconclusive — dependency not observed:**** the WFR client's dependency read does not see a=1.
- ****Inconclusive — successor not observed:**** the MW/WFR observer does not see b=1.

For the main suite, use conservative classification if any scheduled operation errors, including the producer's write. A timed-out write may still have reached replicas; do not assume rollback. Also report MR coverage: how often the first read actually observed a=1, since repeated zero reads do not exercise loss of that update.

5.2 Predictions to register before measurements

| Write/read | RYW prediction under stated assumptions | MR prediction with BLOCKING | MW/WFR interpretation |

|---|---|---|

| ONE/ONE | Stale successful reads are possible | Regression is possible | Missing dependencies are possible |

| QUORUM/ONE | Stale successful reads are possible | Regression is possible | No general per-replica dependency-order claim |

| ONE/QUORUM | No acknowledgement-overlap guarantee: 1+2=3 | Successful quorum reads should not regress | No general per-replica dependency-order claim |

| QUORUM/QUORUM | Expected for acknowledged writes: 2+2>3 | Successful quorum reads should not regress | Quorum overlap alone does not establish causality |

| ALL/ALL | Expected for successful operations | Expected for successful reads | No completed witness expected for this restricted sequential workload |

The overlap argument and read-repair mechanism are documented by Cassandra; applying them to these workloads is our deduction. [1,4] Random routes can land on the same side of a partition and reveal no anomaly, or on opposite sides and reveal one or become unavailable. Do not predict that all ten weak-consistency trials must fail.

6. Exact node-failure experiment

****Scope:**** an abrupt failure of one Cassandra process/container, with its persistent volume retained. This is not a host power failure, disk-loss test, graceful drain, or crash during an in-flight application request. The main histories run after failure detection has stabilized.

6.1 Before each failure episode

1. Verify all three nodes report the intended ring membership, are UN, and respond to direct administrative CQL probes. Check schema agreement.
2. Initialize the 20 fresh trial keys at ALL, using random application routing. Save the successful initialization records.
3. Randomly sample one victim from n1, n2, n3, independently of application routes and case order. Save the victim, container ID, IP, seed and pre-fault state.
4. Record the fault-command start time, then issue an abrupt kill. For example, if the sampled victim is n2:

```
docker compose kill -s SIGKILL n2
```

The executed command must use the sampled victim, not always n2. Do not run `nodetool drain` first; that would make the failure graceful.

6.2 Prove the failure and separate detection from measurement

5. Inspect the stopped container using `docker compose ps -a -q <victim>` and `docker inspect`. Save Running, ExitCode, OOMKilled and FinishedAt. SIGKILL normally gives exit code 137; record the actual state rather than assuming it.
6. Poll `nodetool status` on both surviving nodes every two seconds, up to 90 seconds. Require both to identify the victim as down and the two survivors as up, in two consecutive polls.
7. Independently probe from the client. The routing layer discovers the victim as unreachable; the controller must not directly tell it which endpoint to exclude. Record fault-to-detection time separately from request latency.
8. If the expected fault state is not confirmed, save a setup failure and do not claim that episode's trials ran under the intended condition.

6.3 Run, recover and repeat

9. Execute the 20 model/configuration cases in randomized order. Each read/write draws afresh from the two client-reachable coordinators. Both survivors remain candidates regardless of the configured CL. ONE and QUORUM require at most two replicas; ALL requires three and is expected to be unavailable. [1; application to RF=3]
10. Save all operations, outcomes, node views and end-of-episode container state. Do not retry failed measured operations.
11. Restart the same victim, preserving its volume. Example:

```
docker compose start n2
```

12. Allow up to ten minutes for recovery. Require all three nodes to report the intended live membership, schema agreement, client-side CQL readiness, and a successful fresh-key ALL write/read barrier before starting another block.

Repeat this procedure in ten rounds. Recovery confirms readiness for fresh experiments; with hints disabled, it does not establish that every old divergent key has converged. Fresh initialization at ALL prevents old trial data from contaminating the next block.

The document will include the actual victim sequence, kill/start times, detection and recovery durations, exit states, and the exact commands from every episode.

7. Exact network-partition experiment

****Scope:**** an internode communication partition. All Cassandra processes stay running. The client retains access to all three CQL endpoints, modeling an application able to reach both sides while database nodes cannot communicate across the cut.

7.1 Select and install the cut

1. Begin with verified healthy membership and fresh trial keys initialized at ALL.
2. Randomly select one isolated node. The other two form the majority component. Sample this independently of every data operation; do not choose the isolated node based on where a preceding write landed.
3. Obtain container IPs from Docker inspection; do not hardcode addresses. Save the intended graph. For an n3 victim, it is `{n1,n2} | {n3}`.
4. Create a dedicated `LAB\_FAULT` chain in each database container's filter table and ensure one jump to it at the start of OUTPUT. Container-local NET_ADMIN permits these rules. Do not flush OUTPUT, change the host firewall, or disconnect the client.
5. For every directed edge crossing the cut, install rules matching the peer's destination IP and TCP source or destination ports 7000/7001, with target DROP. Example inside n1 when n3 is the isolated peer:

```
iptables -A LAB_FAULT -d <n3-IP>/32 -p tcp \  
-m multiport --dports 7000,7001 -j DROP  
iptables -A LAB_FAULT -d <n3-IP>/32 -p tcp \  
-m multiport --sports 7000,7001 -j DROP
```

Execute equivalent rules for n2 to n3 and n3 to both n1 and n2. Source-port rules cover the reverse direction of established streams. Inserting the chain ahead of permissive OUTPUT rules ensures established traffic is also subject to the cut. TCP 9042 is not blocked.

7.2 Verify before accepting measurements

6. Save `iptables -S LAB\_FAULT` and verbose numeric packet counters before and after the episode. Verify that every intended cross-cut edge is covered and no within-component edge is blocked.
7. Poll node views, up to 90 seconds. Require the two-node component to see its two members up and the isolated node down; require the isolated node to see itself up and the other two down, in two consecutive polls.
8. Independently demonstrate that all three endpoints answer client-side CQL health queries. All three must be routing candidates. A CL-dependent Unavailable result must not cause the gateway to remove that endpoint.
9. Execute all 20 randomized cases, retaining random candidate draws and actual coordinators. Save increasing DROP counters as evidence that the rules intercepted traffic. Save setup failures separately if the intended graph or reachability is not established.

7.3 Heal safely and repeat

Remove only this experiment's rules or jump. During a change from one partition graph to another, install the union of old and new restrictions first, then remove obsolete restrictions. Never briefly flush the chain and reconnect all nodes between read-repair stages.

After healing, verify the full cluster and perform the fresh-key ALL barrier before the next block. Repeat the main partition episode ten times with newly sampled victims. Record the actual victim sequence, graph, rules, counters, CQL health, node views, and installation/removal timestamps.

A genuine client-versus-server network outage is a different scenario and is outside this baseline. The report must make this topology limitation clear.

8. Redesign of the additional experiments

8.1 Read-repair comparison: ten trials per setting

Use ten episodes, each containing a BLOCKING-table trial and a NONE-table trial on different fresh keys. Randomize table operation order. Application routes are independently random throughout; no read is forced onto the component that can satisfy QUORUM.

1. Initialize both keys at ALL while the cluster is healthy.
2. Fully partition the database nodes from one another while retaining CQL access. Confirm the intended views before issuing a ONE write of $a=1$ to a randomly selected coordinator for each table. This creates a minority version if the write succeeds.
3. Keep the cut in place beyond the configured write-request timeout before allowing communication. Record the configured timeout and actual hold interval, and retain the possibility of delayed messages when interpreting results. Hints remain disabled.
4. Independently select one of the three possible two-node components and allow only that pair to communicate. Confirm the graph and CQL reachability, then perform the first QUORUM read through a fresh random coordinator for each table.
5. Select a different two-node component uniformly from the remaining two choices, independently of data values and previous routes. Change the graph without a fully connected interval. Confirm it, then perform the second QUORUM read through a fresh random coordinator.
6. Heal and verify the cluster; retain all three operations and every topology transition for each trial.

A selected coordinator outside the connected pair cannot satisfy QUORUM: retain that trial as inconclusive. If the first read sees zero, the trial did not expose the minority version; report that coverage condition. If two successful reads return 1 then 0, record a regression. BLOCKING is expected to prevent that regression for successful quorum reads; NONE can permit it. [4]

Randomization can yield no NONE counterexample in ten attempts. That would be a limitation of coverage, not proof that NONE supplies monotonic reads. Do not choose favorable routes after inspecting results. A larger run can be specified in advance and reported as a separate extension.

8.2 Timestamp control: ten trials

With all three nodes healthy, initialize a fresh key at timestamp T . Independently randomize the coordinator for each of these operations:

- ALL write $a=1$ at $T+100$.
- ALL write $a=2$ at $T+50$.
- ALL read a .

Repeat ten times. If both writes are acknowledged, expect $a=1$ because the first write carries the larger mutation timestamp. This deliberately reverses timestamp order while preserving request order. It does not alter the host clocks. [1,5]

The control is excluded from the main consistency-violation rate because it deliberately breaks the increasing-timestamp assumption. Retain errors as inconclusive control outcomes; do not rerun until success.

9. Interpretation boundaries

These experiments test finite, sequential histories on a single-host cluster. They do not prove general causal consistency, model concurrent writers, test cross-partition application dependencies, measure permanent storage loss, or estimate production anomaly rates.

Fault setup is controlled, but operation routing is random and blind to the fault. This is intentional: controlled fault conditions and random application behavior answer complementary parts of the experiment.

Hints are disabled to retain divergence. The report will not claim measurements of convergence under default Cassandra configuration. Successful recovery health checks are not evidence of repair of every old row. Quorum operations may be unavailable; this must be distinguished from returning inconsistent data.

10. Evidence, reporting and validation

10.1 Required records

For each run: design version, UTC start/end, root and derived seeds, software/image versions, cluster configuration, source hashes, routing policy, retry/speculation settings, and measured/setup completion status.

For each trial: unique trial/key/client identifiers, round, episode, model, scenario, configuration, operation order, routing candidates and draws, selected/actual coordinator, mutation timestamp, requested CL, start/end time, monotonic-clock latency, returned values or exact exception, verdict and reason.

For each fault episode: sampled victim or partition graph, command arguments and exit status, installation/kill/detection/recovery timestamps, Docker state, IP mapping, node membership views, CQL health, firewall rules and counters, recovery barrier and cleanup result.

Append records as operations finish so an interrupted run does not lose a whole batch. Save checkpoints atomically. Keep setup failures, aborted episodes and partial measured histories; do not manufacture completion markers. Do not mix old fixed-routing results into the new data set.

10.2 Tables to include in the new report

- The full 3 x 5 x 4 case matrix: attempted, evaluable, violation, no violation observed, and inconclusive counts with reasons.
- Operation success/error counts separately from trial outcomes; latency quantiles separately for successes and failures. Warmup/setup latency is excluded from measured request latency.
- Coordinator counts, write-to-read transition matrices, same/different-node fractions and partition-crossing coverage.
- The exact ten crash victims and ten main partition victims, with detection/recovery timing.
- All twenty read-repair trial sequences and all ten timestamp controls.
- A clear comparison with predictions, including missing counterexamples, unexercised prerequisites and unexpected errors.
- Installation/reproduction commands, limitations, documentation references, and AI-use disclosure.

Ten trials per cell are a minimum demonstration, not a strong statistical sample. Report numerators and denominators. Avoid presenting pooled independent-binomial confidence claims when trials share fault episodes. If estimating uncertainty in a larger extension, account for episode-level clustering and state the sampling assumptions.

10.3 Acceptance checks before publishing results

1. Every main case has at least ten measured attempts; each supplemental type has at least ten attempts. Counts must agree with the case schedule and raw records.
2. Application read/write functions accept no node argument; each operation consumes exactly one independent routing draw. Producer and observer operations use the same policy.
3. The router receives no fault/data-state information. Candidate sets agree with recorded client-side health; partitioned but client-reachable endpoints remain eligible.
4. A fixed seed reconstructs the recorded routing choices for the recorded candidate sets. No test requires empirical frequencies to be perfectly uniform.
5. The checker correctly distinguishes stale reads, regressions, missing dependencies, missing prerequisites, missing rows and request errors. Test it using small synthetic histories, never as substitutes for database measurements.
6. Setup and fault verification precede measurement; commands, exit state and packet counters support the scenario labels. Recovery and cleanup run even after exceptions or interruption.

7. Independent saved-history validation checks ordered operations, fresh-key initialization, timestamp assumptions, requested CLs, coordinator identity, verdicts and source provenance.

8. Both PDFs are regenerated from the new validated run and visually checked. Old results stay clearly labeled as the previous design.

11. Implementation sequence and current gaps

The current repository has a partially updated worker and router, but its orchestration, verifier, tests and reports still describe the earlier interface. It is not yet an integrated randomized suite.

| Step | Files / deliverable | Completion criterion |

|---|---|---|

| 1. Freeze design | This document and a machine-readable run specification | Counts, seeds, routing policy and verdict rules agreed with the documented plan |

| 2. Complete application/gateway separation | src/worker.py, src/routing.py | Random route for every operation; phase health records; no application node argument; no hidden retries |

| 3. Refactor orchestration | scripts/lab.py | Ten rounds; updated worker request schema; random victim/order; measured state polling; cleanup and recovery barriers |

| 4. Update supplemental controls | Worker and runner | Ten randomized trials for each repair setting and timestamp control; all topology phases recorded |

| 5. Update checking | src/checks.py, tests, scripts/verify_results.py | New record schema, prerequisite reasons, missing-row handling, seed replay and evidence validation |

| 6. Smoke-test integration | Separate diagnostic run | Verify startup, one complete history, fault setup/heal, and evidence writing; do not count this as the full measured run |

| 7. Execute the registered run | New timestamped results directory | 600 main + 20 repair + 10 timestamp attempts, or explicit incomplete status |

| 8. Publish artifacts | README, main report, experiment inventory, reproduction archive | Documents use new results and explain routing, SIGKILL, iptables, episode counts and limitations |

Use a default repetition count of 10 and reject lower counts for a full measured run. A distinct smoke-test mode may use fewer attempts but must never emit a full-run completion marker. Provide an optional seed argument for replay and an explicit recovery command that can restart any sampled victim, not only n3.

Do not overwrite the old experiment results. The new results and completion marker must identify the randomized design version, and the report generator must refuse old or incomplete data when producing the redesigned report. Capture the exact source revision or archived source content, not only hashes that cannot later be resolved.

****Planned final outputs:**** the updated executable project; a new results-backed project PDF; a new experiment-list/counts PDF; this detailed methodology; raw JSON/JSONL evidence; and a reproducible source/results archive. No new outcomes are claimed in this plan.

12. Sources and AI assistance

[1] Apache Cassandra, Dynamo: replication, coordinator behavior, mutation versioning and tunable consistency. <https://cassandra.apache.org/doc/stable/cassandra/architecture/dynamo.html>

[2] Apache Cassandra Python driver, policies implementation. <https://github.com/apache/cassandra-python-driver/blob/trunk/cassandra/policies.py>

[3] Apache Cassandra, Hints. <https://cassandra.apache.org/doc/stable/cassandra/managing/operating/hints.html>

[4] Apache Cassandra, Read repair. https://cassandra.apache.org/doc/stable/cassandra/managing/operating/read_repair.html

[5] Apache Cassandra, CQL Data Manipulation.

<https://cassandra.apache.org/doc/stable/cassandra/developing/cql/dml.html>

[6] Terry et al., Session Guarantees for Weakly Consistent Replicated Data (1994).

https://www.cs.cornell.edu/courses/cs734/2000FA/cached%20papers/SessionGuaranteesPDIS_1.html

Documentation checked for the redesign on 12 September 2026. The fault schedules, counts, routing protocol and acceptance checks are proposed experimental choices, not claims made by the database documentation. OpenAI Codex assisted with reviewing the existing code and preparing this plan; randomized trials were not executed as part of preparing this document.